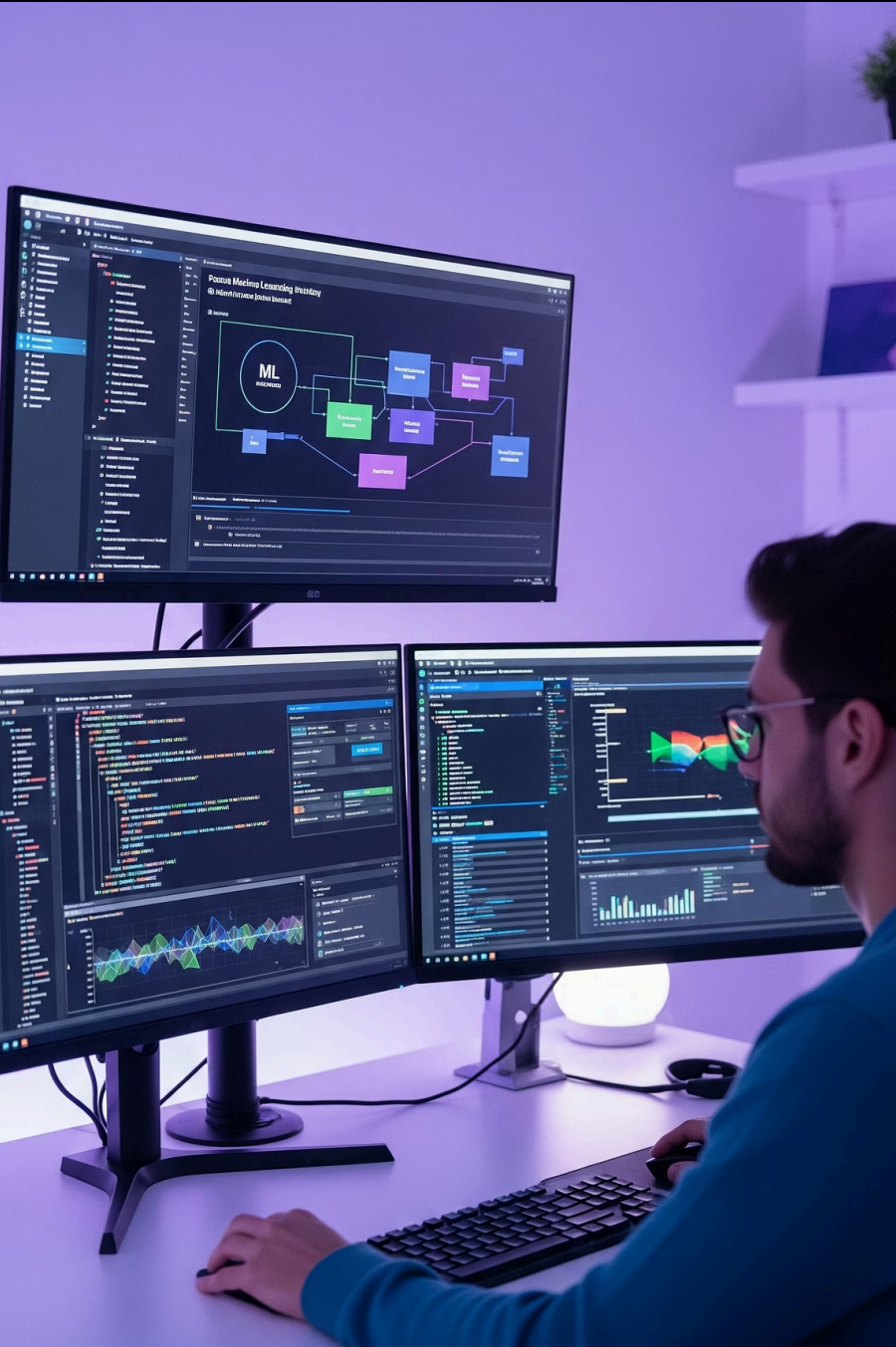




GitLab CI and Jenkins Pipelines for Machine Learning

A practical guide for ML engineers and DevOps practitioners implementing CI/CD for AI workflows in enterprise environments

Why CI/CD Matters for Machine Learning

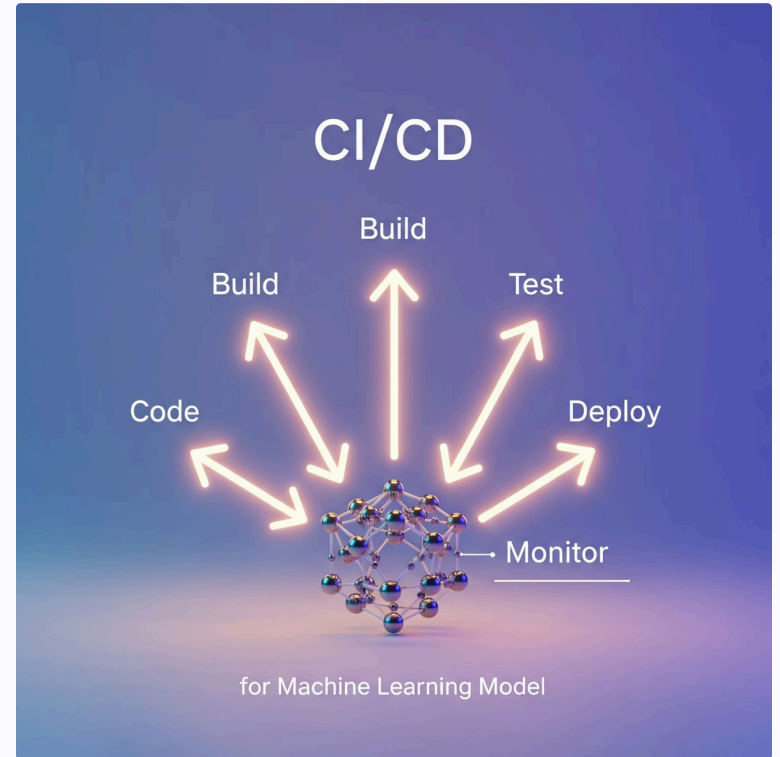
Modern AI projects face unique operational challenges that traditional software doesn't:

Frequent retraining & redeployment cycles as data or requirements evolve

Complex testing requirements for both code and model performance

Resource-intensive workflows requiring specialized hardware

Today we'll focus on **GitLab CI and Jenkins** - two industry-leading tools powering enterprise ML pipelines.



GitLab CI Overview

GitLab CI offers a comprehensive, integrated approach to ML pipeline automation:

Native Integration

Seamlessly works with GitLab repositories - no additional configuration needed

YAML-Based Workflows

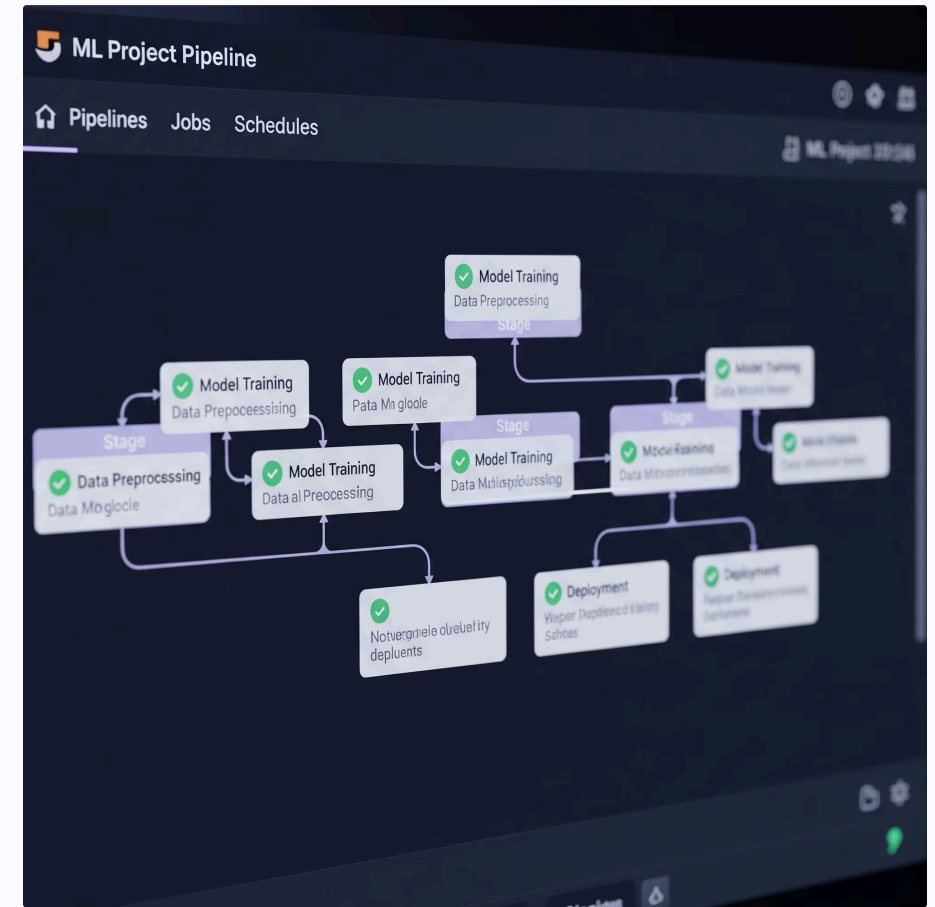
Pipelines defined in .gitlab-ci.yml - simple, declarative, and version-controlled

Runners Architecture

Flexible execution environments that can scale from local machines to enterprise clusters

Security Focus

Built-in scanning for vulnerabilities in dependencies and container images



GitLab CI for Machine Learning



GPU Support

Configure GPU-enabled runners for compute-intensive model training



ML Pipeline Stages

Data validation → Training → Evaluation → Deployment



Container Registry

Built-in storage for versioned, Dockerized ML applications



Artifact Management

Store and version model weights, logs, and metrics

GitLab CI's integration with **Kubernetes and cloud platforms** makes it particularly valuable for organizations that have embraced cloud-native infrastructure for their ML operations.

GitLab CI Example Workflow

```
stages:
  - build
  - train
  - deploy

train-model:
  stage: train
  image: pytorch/pytorch:latest
  script:
    - pip install -r requirements.txt
    - python train.py
  artifacts:
    paths:
      - "model.pth"
```

This simple configuration:

- Defines a multi-stage ML pipeline
- Uses PyTorch's official container
- Executes the training script
- Preserves the trained model as an artifact

Extended Capabilities

Conditional pipelines based on commit messages or file changes

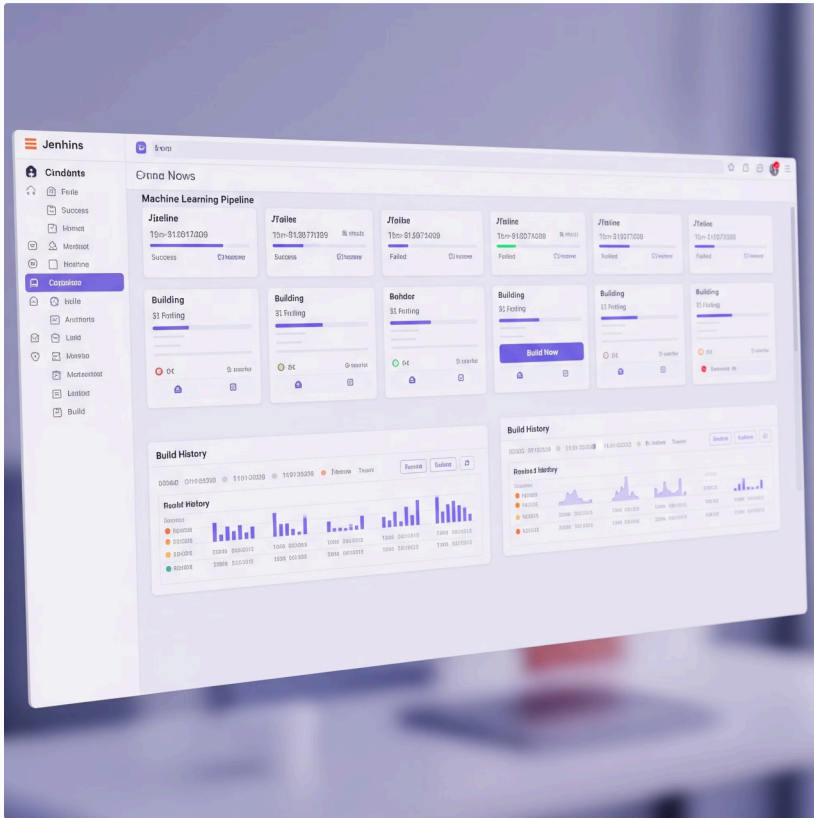
Scheduled retraining on fixed intervals

Dynamic environments for feature-specific model testing

Multi-architecture builds for edge deployment



Jenkins Overview



As the veteran in the CI/CD space, Jenkins brings maturity and flexibility to ML workflows:

Open-Source Legacy

Battle-tested since 2011 with widespread enterprise adoption

Extensive Plugin Ecosystem

2000+ plugins including specialized ML and data science tools

Groovy-Based Pipelines

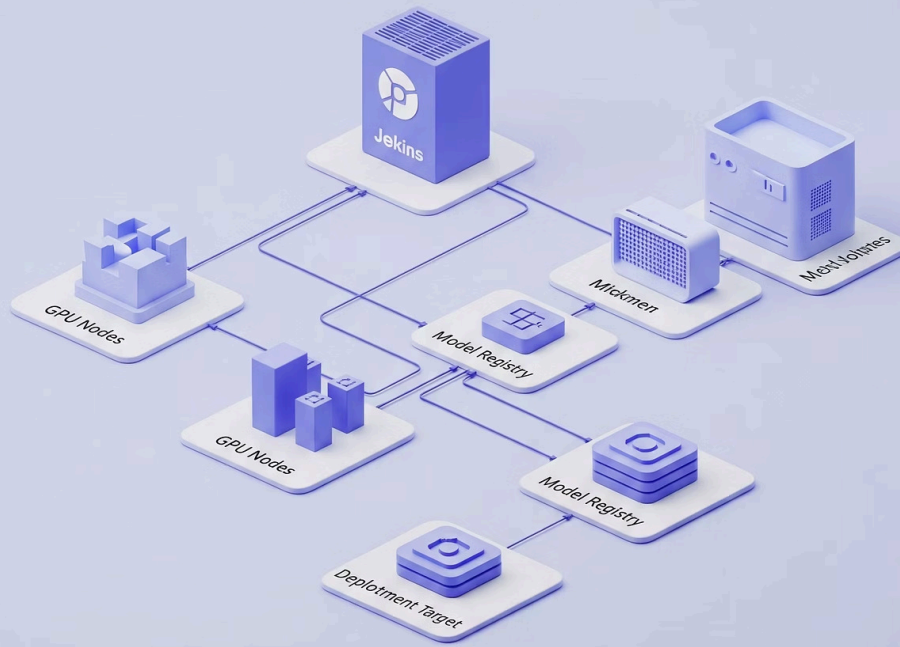
Powerful scripting capabilities via Jenkinsfile configuration

Enterprise Adaptability

Highly customizable for complex infrastructure requirements

Jenkins remains **dominant in hybrid environments** where on-premises and cloud resources need orchestration.

Jenkins for ML Projects



Source Integration

Connects with all major Git providers and triggers builds on data or code changes



Compute Orchestration

Supports GPU acceleration via custom agent configuration for training workloads



Deployment Automation

Triggers model deployment after validation metrics meet thresholds

Jenkins' flexibility makes it particularly valuable in **regulated industries like finance, healthcare, and telecommunications** where compliance requirements add complexity to ML infrastructure.

Jenkins Pipeline Example

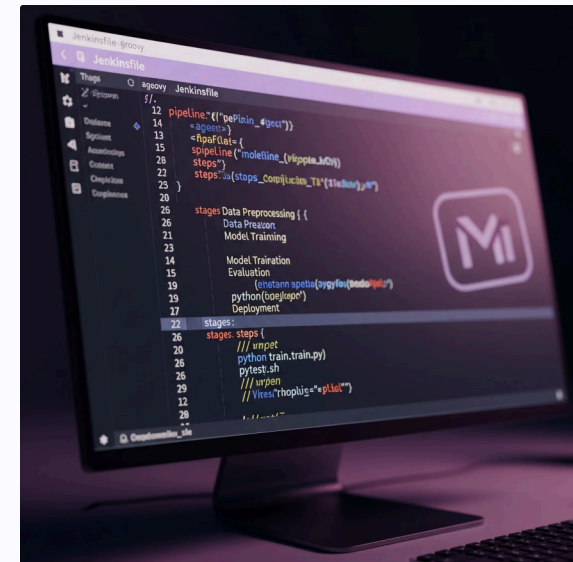
```
pipeline {
  agent {
    docker {
      image 'tensorflow/tensorflow:latest'
    }
  }
  stages {
    stage('Train') {
      steps {
        sh 'pip install -r requirements.txt'
        sh 'python train.py'
      }
    }
    stage('Deploy') {
      steps {
        sh './deploy.sh'
      }
    }
  }
}
```

This Jenkinsfile:

- Uses TensorFlow's Docker image as the execution environment
- Defines distinct training and deployment stages
- Executes shell commands for each pipeline phase

Advanced Jenkins Patterns

- Parameterized builds for experimenting with hyperparameters
- Parallel execution for simultaneous model training
- Shared libraries for ML-specific pipeline functions
- Integration with notification systems for model drift alerts



GitLab CI vs Jenkins: Feature Comparison

Feature	GitLab CI	Jenkins
Hosting Options	SaaS + self-hosted	Self-hosted only
Configuration	YAML (declarative)	Groovy DSL (programmatic)
GPU Support	Via GitLab runners	Via custom agents
Ease of Setup	Simple, built-in	Flexible, but complex
ML Integration	Container registry, artifacts	Rich plugin ecosystem
Best Suited For	Cloud-native ML teams	Enterprise hybrid infrastructure

The choice between these platforms often comes down to **existing infrastructure investments** and organizational preferences around tooling philosophy.

Common Challenges

GitLab CI Challenges

GPU Runner Setup: Significant initial configuration cost for specialized hardware

Resource Limits: Default SaaS runners have execution time constraints

Cost Management: Potential for unexpected expenses with self-managed runners

Jenkins Challenges

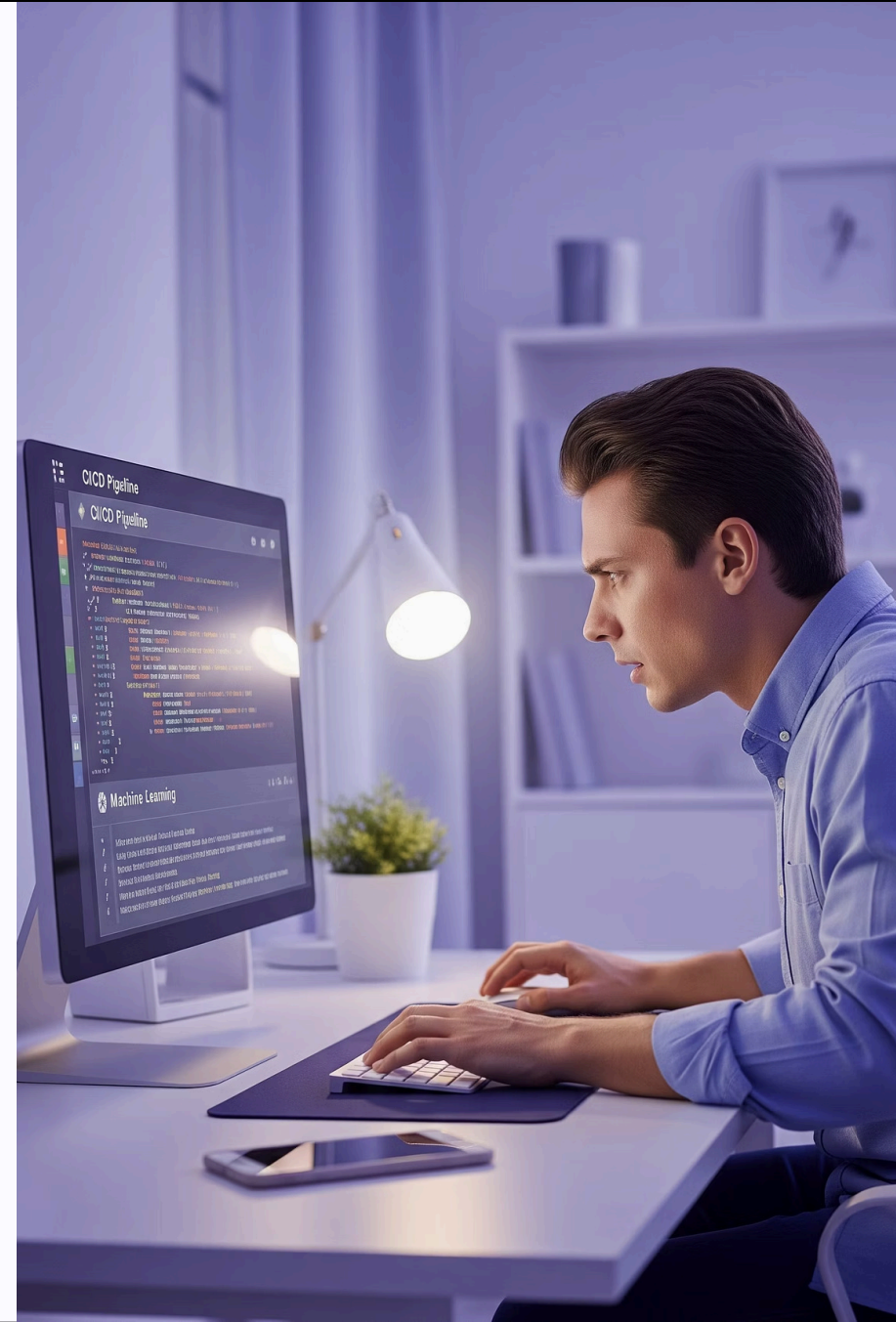
Maintenance Overhead: Requires dedicated DevOps resources

Plugin Management: Version compatibility issues between plugins

Scaling Complexity: Non-trivial to scale for large ML workloads

Shared Challenges

Both platforms require thoughtful integration with **experiment tracking tools** like MLflow or Weights & Biases to enable full ML lifecycle management and reproducibility.



Key Takeaways

2

Leading Platforms

GitLab CI and Jenkins represent two different but powerful approaches to ML automation

4

Critical Factors

Infrastructure strategy, team expertise, compliance needs, and scaling requirements should drive your choice

3

Integration Points

Success depends on thoughtful connections to experiment tracking, model registry, and monitoring tools

Decision framework: Choose [GitLab CI](#) for cloud-native, modern ML teams prioritizing simplicity and integration. Choose [Jenkins](#) for enterprise organizations with complex infrastructure and legacy system requirements.

Remember that the right CI/CD implementation can dramatically reduce the friction between model development and production deployment, accelerating your ML team's ability to deliver business value.